Research Project

A Low-Latency FFT Pricing Architecture: Beyond Finite Differences and Analytical Greeks in C++

Edoardo Mocchi, Lorenzo Pirozzi*, Pietro Bottan, Simone Coppetti, Achille Galante, Simone Lorello and David Piana

BlackSwan Quants, Milan, Italy

This manuscript was compiled on April 7, 2026

Abstract

This paper presents the design and implementation of a low-latency, hybrid option pricing engine leveraging the Carr-Madan Fast Fourier Transform (FFT) framework. While traditional pricing engines based on Partial Differential Equations (PDEs) or Monte Carlo simulations suffer from performance bottlenecks and rigid architectures, our FFT approach completely decouples the numerical integration engine from the underlying stochastic dynamics. To overcome the computational latency of pure Python implementations during intensive calibration procedures, we developed a hardware-sympathetic architecture: the core execution loops, complex arithmetic, and FFT algorithms are implemented natively in C++ and interfaced seamlessly via Pybind11. By requiring only the analytical characteristic function of the log-price, the architecture supports advanced market models—ranging from Black-Scholes and Heston stochastic volatility to Levy-driven processes like Merton Jump-Diffusion (MJD) and Variance Gamma (VG). Crucially, model calibration is formulated directly in implied volatility (IV) space rather than price space, ensuring robust gradient dynamics across all moneyness regimes. Furthermore, we extend the framework to the analytical frequency-domain derivation of the Greeks (Delta, Gamma, Vega). This allows for the simultaneous, high-precision computation of entire sensitivity surfaces across thousands of strikes in $\mathcal{O}(N \log N)$ time, bypassing the computational overhead and numerical noise of traditional finite-difference bumping techniques.

Keywords: *Quantitative Development; Fast Fourier Transform; Hybrid Architecture; Low-Latency; Heston Model; Jump-Diffusion; Analytical Greeks; Risk Management*

1. INTRODUCTION

The evolution of quantitative finance has driven a paradigm shift from the idealized Black-Scholes framework toward sophisticated dynamics capable of capturing empirical market phenomena, such as volatility smiles and fat tails. Models like Heston stochastic volatility, Merton Jump-Diffusion (MJD), and Variance Gamma (VG) are now industry standards. However, these advanced models typically lack closed-form probability density functions, rendering standard analytical pricing impossible and making Monte Carlo simulations computationally expensive, especially for real-time surface calibration.

Fortunately, while their explicit densities are unknown, these models possess analytically tractable characteristic functions derived via affine jump-diffusion theory and Riccati equations [5]. The Carr-Madan Fast Fourier Transform (FFT) pricing methodology [4] leverages this feature, shifting the valuation problem from the time domain to the frequency domain.

In this project, developed by *BlackSwan Quants*, we establish a rigorous balance between mathematical analysis and high-performance Quantitative Development. To overcome the computational latency of pure Python implementations during intensive calibration procedures, we developed a hybrid architecture: the core execution loops, complex arithmetic, and FFT algorithms are implemented natively in C++ and interfaced seamlessly via Pybind11. In our hardware-sympathetic design, the FFT numerical integrator acts as an agnostic engine requiring only the analytical characteristic kernel, ensuring extreme scalability. Furthermore, targeting implied volatility (IV) directly in the loss function prevents deep In-The-Money artifacts from dominating the optimization landscape.

Crucially, we rigorously extend the framework to the analytical frequency-domain derivation of the Greeks (Delta, Gamma, Vega). By differentiating the characteristic function directly in the complex plane, our engine recovers entire sensitivity surfaces simultaneously in $\mathcal{O}(N \log N)$ time.

*Corresponding author: lorenzo.pirozzi@mail.polimi.it

2. LITERATURE REVIEW: ALTERNATIVE METHODOLOGIES

Pricing an entire option surface entails computing values for M distinct strikes. To justify our architectural focus on the FFT approach, we compare it against the primary numerical alternatives for models lacking closed-form densities.

2.1. Monte Carlo Simulation

By the Central Limit Theorem, the statistical error of a Monte Carlo estimator decays as $\mathcal{O}(N_{\text{sim}}^{-1/2})$, where N_{sim} is the number of simulated paths [6]. Pricing M strikes independently thus implies a complexity of $\mathcal{O}(M \cdot \epsilon^{-2})$ to achieve a target accuracy ϵ.

To construct a methodologically fair benchmark, we implement a Control Variates scheme [2] on the full Heston dynamics. The Euler-Maruyama discretization of the Heston SDE and a shadow Black-Scholes path are driven by the *same* pseudo-random Brownian increments Z_1, inducing maximal correlation between the two payoffs. The resulting variance-reduced estimator is:

$$C_{\text{Heston}}^{\text{CV}} = \mathbb{E}\big[\max(S_T^{\text{Heston}} - K, 0)\big] - \beta\Big(\mathbb{E}\big[\max(S_T^{\text{BS}} - K, 0)\big] - C_{\text{analytical}}^{\text{BS}}\Big) \tag{1}$$

where $\beta = 1$ is the control coefficient [6]. This construction yields stable price estimates with as few as $N_{\text{sim}} = 10{,}000$ paths.

Despite this variance reduction, a fundamental asymmetry in computational structure persists. With N_{sim} fixed independently of M, the true runtime takes the form:

$$T_{\text{MC}}(M) \approx \underbrace{N_{\text{sim}} \cdot N_{\text{steps}}}_{\text{path generation overhead}} + N_{\text{sim}} \cdot M \tag{2}$$

The dominant constant—comprising path simulation, RNG calls, and array allocation—places the effective intercept of T_{MC} several orders of magnitude above the FFT baseline, so that the asymptotic

This project aims to implement and document an option pricing engine based on Fourier methods (Carr-Madan FFT) for European options. The core objective is to value options utilizing the characteristic function of the log-price, addressing scenarios where the explicit density function is unavailable.

$\mathcal{O}(M)$ slope is never reached in any computationally relevant regime. Furthermore, maintaining constant relative accuracy as the strike grid is refined forces $N_{\text{sim}} = \mathcal{O}(M^2)$, since preserving a fixed signal-to-noise ratio at resolution $\Delta K \sim 1/M$ requires the standard error $\epsilon \sim 1/\sqrt{N_{\text{sim}}}$ to scale accordingly. The true complexity of a precision-consistent Monte Carlo surface is therefore $\mathcal{O}(M^3)$, rendering it entirely impractical for dense grid generation.

Finally, Monte Carlo remains notoriously inefficient for Greeks via finite differences: shifting the initial spot and recomputing the expectation amplifies estimator variance by a factor proportional to $1/\Delta S^2$, requiring a disproportionate increase in N_{sim} to recover stable gradients [2].

2.2. Direct Quadrature and the COS Method

Evaluating the inverse Fourier integral individually for each strike via numerical quadrature requires N evaluations of the characteristic function per strike, leading to a quadratic global complexity of $\mathcal{O}(M \cdot N)$. Advanced alternatives, such as the Fourier-Cosine Series Expansion (COS method) [11], achieve exponential convergence for smooth density functions and are currently considered an industry standard for single-asset pricing. Nevertheless, while the COS method is highly efficient for isolated option pricing, the FFT architecture remains fundamentally superior when computing entire option surfaces and sensitivity grids simultaneously.

2.3. The Fast Fourier Transform (FFT) Paradigm

By discretizing the integral on a regular grid and leveraging the FFT algorithm [1], the Carr-Madan approach ensures the N evaluations of the characteristic function are computed exactly once. The simultaneous extraction of $M = N$ prices requires a single execution of the DFT, reducing the overall complexity to:

$$\mathcal{C}_{FFT} = \mathcal{O}(N \log_2 N) \tag{3}$$

This logarithmic reduction is the core mathematical driver that enables real-time surface generation. Computing Greeks in the frequency domain requires only an algebraic modulation of the base integrand $\psi_T(v)$, completed in $\mathcal{O}(N)$ time, followed by a single additional IFFT execution to recover the spatial sensitivity grid. The marginal cost per Greek surface is therefore $\mathcal{O}(N \log N)$, and crucially, it completely bypasses the re-evaluation of the characteristic function—which involves expensive complex logarithms and principal branch resolutions—required by standard finite-difference bumping techniques.

Figure 1 empirically validates these claims. Even against the methodologically rigorous Control Variates baseline of Section 2.1, the MC curve exhibits an $\mathcal{O}(N)$ slope whose dominant overhead constant sustains a two-order-of-magnitude gap with respect to the FFT across the entire operationally relevant range. When the precision-consistency argument is factored in, elevating the true MC complexity to $\mathcal{O}(N^3)$, the FFT advantage becomes insurmountable.

3. PROJECT ROADMAP & ARCHITECTURE

The project was structured around modern quantitative development practices, prioritizing modularity, speed, and numerical precision:

- **Phase 1: Hybrid Architecture Design.** Definition of base classes and interfaces. Separation of concerns between the `StochasticProcess` module (housing the analytical characteristic functions) and the compiled `FFTIntegrator` engine.
- **Phase 2: Core Engine Implementation.** Coding and vectorization of the Carr-Madan FFT formula in C++, incorporating optimal grid spacing (η) and Nyquist relation mapping for strike generation.

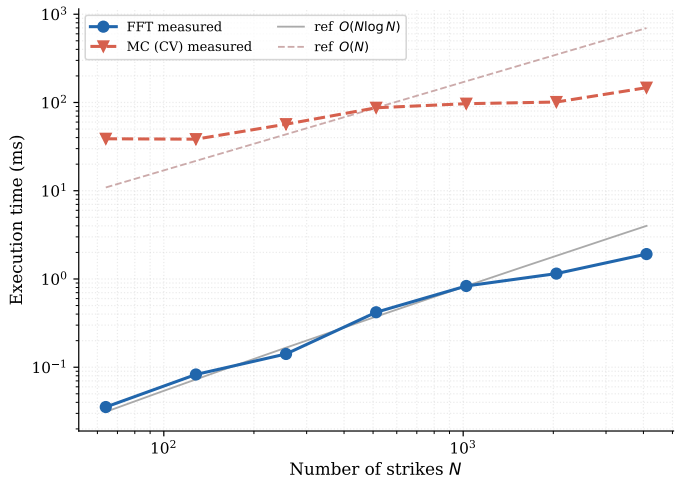

Figure 1. Execution time scaling for the Carr-Madan FFT engine vs. a Control Variates Monte Carlo estimator on the Heston model ($N_{\text{sim}} = 10{,}000$ paths, Euler-Maruyama discretization) as a function of the number of strikes N. The FFT curve follows the predicted $\mathcal{O}(N \log N)$ slope. Despite variance reduction, the MC baseline tracks $\mathcal{O}(N)$ with a constant overhead that sustains a two-order-of-magnitude gap in the full operationally relevant regime.

- **Phase 3: Exotic Dynamics Integration.** Extending the process library beyond Black-Scholes to encompass the Heston model, Merton Jump-Diffusion (MJD), and Variance Gamma (VG), demonstrating the plug-and-play nature of the engine.
- **Phase 4: Analytical Greeks Engine.** Mathematical derivation and implementation of frequency-domain partial derivatives to compute Delta, Gamma, and Vega directly via inverse FFT, avoiding finite-difference approximations.
- **Phase 5: Performance Benchmarking.** Rigorous computational profiling of the engine against closed-form benchmarks and Monte Carlo equivalents, proving the superiority of the FFT approach in generating massive option and risk surfaces.

4. METHODOLOGY: THE LOW-LATENCY FFT ENGINE

4.1. Pricing Framework

Following the foundational work of [4], let $C_T(k)$ denote the value of a European call option with log-strike $k = \ln(K)$ and maturity T. The direct Fourier transform of $C_T(k)$ with respect to k does not exist in the Lebesgue sense. As $k \to -\infty$ (strike goes to zero), the call option is deep in-the-money and its price asymptotically approaches the spot price S_0, violating the $L^1(\mathbb{R})$ integrability requirement [9]. To enforce square-integrability, a damping factor $\alpha > 0$ is introduced to define the exponentially dampened option price:

$$c_T(k) = e^{\alpha k} C_T(k) \tag{4}$$

The Fourier transform of this dampened price, denoted as $\psi_T(v)$, can be expressed purely in terms of the characteristic function of the log-price under the risk-neutral measure, $\phi_T(u) = \mathbb{E}^{\mathbb{Q}}[e^{iuS_T}]$, yielding:

$$\psi_T(v) = \frac{e^{-rT}\phi_T(v - (\alpha+1)i)}{\alpha^2 + \alpha - v^2 + i(2\alpha+1)v} \tag{5}$$

where r is the risk-free rate.

4.2. Architectural Decoupling and FFT Discretization

From a software engineering perspective, the elegance of the Carr-Madan approach lies in its strict separation of concerns. The pricing engine acts purely as an `FFTIntegrator`, entirely agnostic to the underlying stochastic dynamics. It only requires a standardized interface to

retrieve $\phi_T(u)$. Call prices are then recovered via the inverse Fourier transform:

$$C_T(k) = \frac{e^{-\alpha k}}{\pi} \int_0^\infty e^{-ivk} \psi_T(v)\, dv \tag{6}$$

To compute this efficiently, we shift from continuous integration to a discrete summation over a fine grid. We truncate the upper limit of integration to $a = N\eta$, evaluating the integrand at N discrete frequencies $v_j = (j-1)\eta$ for $j = 1, \dots, N$. Simultaneously, we define a regular grid for the log-strikes: $k_m = -b + (m-1)\lambda$ for $m = 1, \dots, N$.

To apply the Cooley-Tukey Fast Fourier Transform [1], the frequency step η and the strike step λ must strictly satisfy the Nyquist relation: $\lambda\eta = 2\pi/N$. It is crucial to note that this inversely couples the two grids: the FFT evaluates prices for a rigid, predetermined vector of strikes k_m [9]. The practitioner cannot arbitrarily query specific strikes, but rather obtains an equidistant grid governed by the choice of η and N.

Substituting v_j and k_m into the integral and applying Simpson's rule—which guarantees fourth-order numerical convergence for sufficiently smooth integrands [13]—the price array is evaluated as:

$$C_T(k_m) \approx \frac{e^{-\alpha k_m}}{\pi} \sum_{j=1}^{N} e^{-i\frac{2\pi}{N}(j-1)(m-1)} e^{ibv_j} \psi_T(v_j) W_j \eta \tag{7}$$

where $W_j = \frac{1}{3}[3 + (-1)^j - \delta_{j-1}]$. This transition from a continuous integral over an infinite domain to a finite discrete sum introduces two distinct numerical artifacts: a truncation error and a discretization error. These will be formally bounded in Section 6.3.

4.3. The Damping Parameter α: Analytical Constraints

While the damping factor $\alpha > 0$ guarantees L^1 integrability, its selection is heavily constrained by the analytical properties of the underlying stochastic process.

The Fourier integral $\psi_T(v)$ evaluates the characteristic function at the complex argument $z = v - i(\alpha + 1)$. For this expectation to be mathematically well-defined, the horizontal contour must lie strictly within the strip of analyticity $\mathcal{S}_X$ of the characteristic function. According to affine jump-diffusion theory [10], this strip is bounded above by a critical moment explosion threshold $k_{max}(\Theta, T)$, which is explicitly dependent on the model parameters Θ and the time to maturity T. Consequently, the operational constraint for the FFT engine is:

$$\alpha + 1 < k_{max}(\Theta, T) \tag{8}$$

Imposing a static, arbitrarily chosen α (e.g., $\alpha = 1.5$) creates a severe vulnerability during market calibration. When the optimizer explores parameter regimes characterized by high stochastic noise—specifically, a high volatility-of-volatility (ξ) or highly negative correlation (ρ)—the analyticity strip narrows aggressively, and k_{max} drops.

If the static α violates the boundary, the integration contour exits the valid probability domain. The algorithm evaluates the analytic continuation of the Riccati equations, quietly crossing branch cuts and producing finite, yet probabilistically meaningless prices. This *silent failure* [12] traps the optimizer in spurious local minima, severely distorting the estimated parameters. To ensure global topological consistency, our engine implements a dynamic damping factor, strictly bounding $\alpha \in [\epsilon, k_{max}(\Theta, T) - 1)$ at each iteration of the optimization routine.

5. ANALYTICAL GREEKS ENGINE VIA FFT

Standard industry practice often relies on finite difference methods ("bump-and-revalue") to compute Greeks, which compounds computational time and introduces numerical instability. While advanced alternatives exist for Monte Carlo, such as the Pathwise Derivative or Likelihood Ratio methods [2], they require complex model-specific derivations and remain computationally intensive for full surfaces. We overcome this by analytically differentiating the characteristic functions.

The interchange of the differential operator and the integration sign is mathematically justified by the Leibniz integral rule, predicated on Lebesgue's Dominated Convergence Theorem. Provided that the integrand $\psi_T(v)$ and its partial derivatives are continuous and bounded by an integrable dominating function, the swap is rigorous. For affine models like Heston, the exponential decay of the characteristic function at infinity [10] ensures this condition is strictly met.

By applying this property, the Greek integrands are obtained via a simple $\mathcal{O}(N)$ complex modulation of the pricing vector. A subsequent FFT inversion generates the entire risk surface in $\mathcal{O}(N \log N)$.

5.1. Analytic Derivation of the Delta

Let X_T denote the process increments such that the log-price is $s_T = \ln S_0 + X_T$. Because X_T is independent of the initial state S_0, the characteristic function factorizes:

$$\phi_T(u) = \mathbb{E}[e^{ius_T}] = S_0^{iu}\mathbb{E}[e^{iuX_T}] = S_0^{iu}\phi_T^X(u) \tag{9}$$

Differentiating directly with respect to S_0 yields:

$$\frac{\partial \phi_T(u)}{\partial S_0} = iuS_0^{iu-1}\phi_T^X(u) = \frac{iu}{S_0}\phi_T(u) \tag{10}$$

Recall that the integrand for the price is $\psi_T(v) = C \cdot \phi_T(u)$, where $C = e^{-rT}/(\dots)$ is a scaling factor independent of S_0, and the complex argument is $u = v - (\alpha + 1)i$. Applying the chain rule, the partial derivative of the dampened transform is:

$$\frac{\partial \psi_T(v)}{\partial S_0} = C \cdot \left[\frac{iu}{S_0}\phi_T(u)\right] = \frac{i(v - (\alpha+1)i)}{S_0}\psi_T(v) \tag{11}$$

Defining $\psi_\Delta(v) = \frac{\partial \psi_T(v)}{\partial S_0}$, we obtain the linear modulation:

$$\psi_\Delta(v) = \frac{\alpha + 1 + iv}{S_0}\psi_T(v) \tag{12}$$

The Delta is then efficiently computed via inverse FFT:

$$\Delta(k) = \frac{e^{-\alpha k}}{\pi} \int_0^\infty e^{-ivk}\psi_\Delta(v)\, dv \tag{13}$$

5.2. Analytic Derivation of the Gamma

The option Gamma, $\Gamma = \frac{\partial^2 C_T}{\partial S_0^2}$, is derived by differentiating (10) a second time with respect to the spot price:

$$\frac{\partial^2 \phi_T(u)}{\partial S_0^2} = \frac{iu(iu-1)}{S_0^2}\phi_T(u) = -\frac{u^2 + iu}{S_0^2}\phi_T(u) \tag{14}$$

Substituting $u = v - (\alpha + 1)i$, the modulated integrand for the Gamma becomes:

$$\psi_\Gamma(v) = \frac{1}{S_0^2}\left[(iv + \alpha + 1)(iv + \alpha)\right]\psi_T(v) \tag{15}$$

Which is subsequently recovered via the inverse FFT:

$$\Gamma(k) = \frac{e^{-\alpha k}}{\pi} \int_0^\infty e^{-ivk}\psi_\Gamma(v)\, dv \tag{16}$$

5.3. Analytic Derivation of the Vega (Black-Scholes Case)

While Delta and Gamma derive structurally from the homogeneous properties of the option price with respect to the underlying state vari-

able S_0 (operating as model-free dynamics in the transformed space), Vega ($\mathcal{V} = \frac{\partial C_T}{\partial \sigma}$) is strictly model-dependent. To demonstrate the architecture's capability to embed arbitrary parameter sensitivities, we differentiate the Geometric Brownian Motion characteristic function with respect to σ:

$$\frac{\partial \phi_T^{BS}(u)}{\partial \sigma} = -\sigma T(u^2 + iu)\phi_T^{BS}(u) \tag{17}$$

The Fourier transform of the Vega, $\psi_{\mathcal{V}}(v)$, is obtained by modulating the pricing integrand:

$$\psi_{\mathcal{V}}(v) = -\sigma T\left[(iv + \alpha + 1)(iv + \alpha)\right]\psi_T(v) \tag{18}$$

And inverted via FFT:

$$\mathcal{V}(k) = \frac{e^{-\alpha k}}{\pi}\int_0^\infty e^{-ivk}\psi_{\mathcal{V}}(v)\,dv \tag{19}$$

6. EMPIRICAL VALIDATION ON REAL DATA

To validate the robustness of our architecture and the critical importance of the adaptive α mechanism, the engine was calibrated against real market options data. The inverse problem of recovering parameters from market quotes is notoriously ill-posed and characterized by highly non-convex loss landscapes [8]. Consequently, optimization algorithms (such as the iterative Levenberg-Marquardt scheme) require thousands of rapid objective function evaluations. The hybrid C++/Python architecture developed in this project is strictly necessary to prevent computational bottlenecks during this phase, ensuring that the optimizer can traverse the complex parameter space without being hindered by interpreter latency.

6.1. Dataset and Numerical Implementation

Market data is sourced from live S&P 500 ETF (SPY) options retrieved via the `yfinance` API. To ensure a statistically meaningful calibration, the dataset is filtered along two dimensions:

- **Liquidity filter:** Only contracts with daily volume > 10 are retained, eliminating stale quotes and illiquid OTM options whose bid-ask spreads dominate the pricing signal.
- **Moneyness filter:** The calibration window is restricted to $K/S_0 \in [0.94, 1.10]$, covering the liquid near-the-money region while discarding deep-OTM strikes where the implied volatility becomes numerically unstable.

A key design decision is the choice of **loss metric**. Rather than minimizing the mean squared error (MSE) in price space, our calibration targets **implied volatility space**:

$$\mathcal{L}(\theta) = \frac{1}{M}\sum_{i=1}^{M}\left(\sigma_i^{mkt} - \sigma_i^{model}(K_i, T; \theta)\right)^2 \tag{20}$$

This is crucial because price-space MSE is dominated by ITM options, which carry much greater intrinsic value but convey no information about volatility dynamics. The IV-space loss treats all strikes symmetrically and is invariant to the overall level of the spot price S_0.

The characteristic functions are evaluated under a **log-return normalization** ($S_0 = 1$), operating on log-moneyness $k = \ln(K/S_0)$ rather than the raw log-strike $\ln(K)$. This eliminates the high-frequency oscillatory phase $e^{iu\ln S_0}$ from the Fourier integrand, which would otherwise induce aliasing artifacts for large spot prices $S_0 \gg 1$. The dividend yield $q = 1.35\%$ — characteristic of SPY — is included in the risk-neutral drift of all models to ensure correct forward pricing and Put-Call parity consistency.

Global optimization is performed via **Differential Evolution** (DE) [3], a population-based metaheuristic that does not require gradient information and exhibits superior robustness on non-convex, multimodal loss landscapes compared to local methods such as Levenberg-Marquardt or BFGS. The Heston model additionally enforces the **Feller condition** $2\kappa\theta > \sigma_v^2$ via a penalty term to prevent the variance process from reaching zero.

6.2. Model Performance: Diffusion vs. Pure Jumps at Short Maturities

As highlighted in Figure 2, the Variance Gamma (VG) model systematically outperforms the Heston model in the near-term calibration window ($T \approx 0.013$ years), achieving an RMSE of 0.767 against Heston's 2.521. This result might seem counter-intuitive given that Heston possesses more degrees of freedom (5 parameters) compared to VG (3 parameters). However, the discrepancy is deeply rooted in the analytical structure of the underlying stochastic processes as $T \to 0$.

The Heston model relies on a continuous diffusion process to drive variance. At ultra-short maturities, the variance process simply lacks the required time to diffuse away from its initial state v_0. Consequently, the probability of extreme tail events shrinks, the volatility smile flattens, and the model mathematically collapses toward Black-Scholes.

Conversely, Variance Gamma is an infinite-activity pure jump process. The state variable does not diffuse continuously but moves entirely via instantaneous jumps. This mathematical formulation allows VG to instantaneously price the risk of extreme market gaps (e.g., overnight gaps or "crash phobia") regardless of how close the option is to expiration. For highly liquid, short-dated index options like SPY, the market heavily prices this jump risk, making the parsimonious VG model structurally superior to pure diffusion models in the short-term regime.

6.3. Numerical Precision and Error Bounds

Beyond computational speed, formulating analytical derivatives in the frequency domain eliminates the need for numerical finite differences, completely circumventing the truncation and round-off errors inherent in approximating derivatives via state perturbation.

Focusing on the pricing integral itself, the total numerical error $\mathcal{E}_{total}$ introduced by the FFT pricer can be analytically decomposed into two strictly distinct components: the truncation error and the discretization error. Let $a = N\eta$ be the upper limit of the integration grid. The exact price is given by $C(k) = \mathcal{I} = \int_0^\infty f(v)\,dv$, while the FFT computes the finite sum $\hat{\mathcal{I}}$.

6.3.1. Truncation Error ($\mathcal{E}_{trunc}$)

This error originates from replacing the continuous integral over $[0, a]$ with a discrete summation. Because our engine implements Simpson's rule, the error bound is governed by the fourth derivative of the integrand. According to standard numerical analysis limits [13]:

$$\mathcal{E}_{trunc} = \left|\int_a^\infty e^{-ivk}\psi_T(v)\,dv\right| \le \int_a^\infty |\psi_T(v)|\,dv \tag{21}$$

For processes with regular characteristic functions like Heston, provided that the damping factor α is chosen strictly within the strip of analyticity, the integrand exhibits exponential decay [10]. Consequently, $\mathcal{E}_{trunc}$ can be driven to machine precision by choosing a sufficiently large upper bound a.

6.3.2. Discretization Error ($\mathcal{E}_{disc}$) and the Nyquist Trade-off

This error, often referred to as aliasing, originates from replacing the continuous integral over $[0, a]$ with a discrete summation over N points separated by η. Because our engine implements Simpson's rule, the error bound is governed by the fourth derivative of the integrand. According to standard numerical analysis limits [13]:

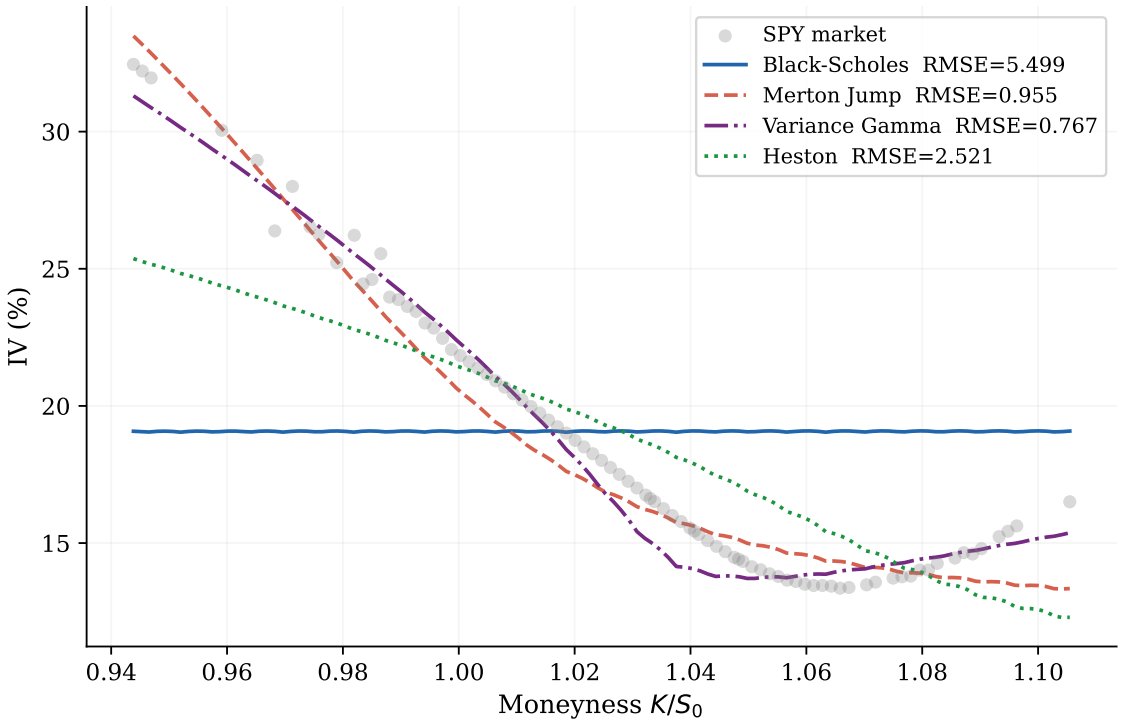

Figure 2. Implied volatility smile calibrated on live SPY market data (S&P 500 ETF). Four models are simultaneously fitted: Black-Scholes (flat by construction), Merton Jump-Diffusion, Variance Gamma, and Heston stochastic volatility. The dividend yield $q = 1.35\%$ is included in the forward. RMSE values in implied volatility space are reported in the legend for each model.

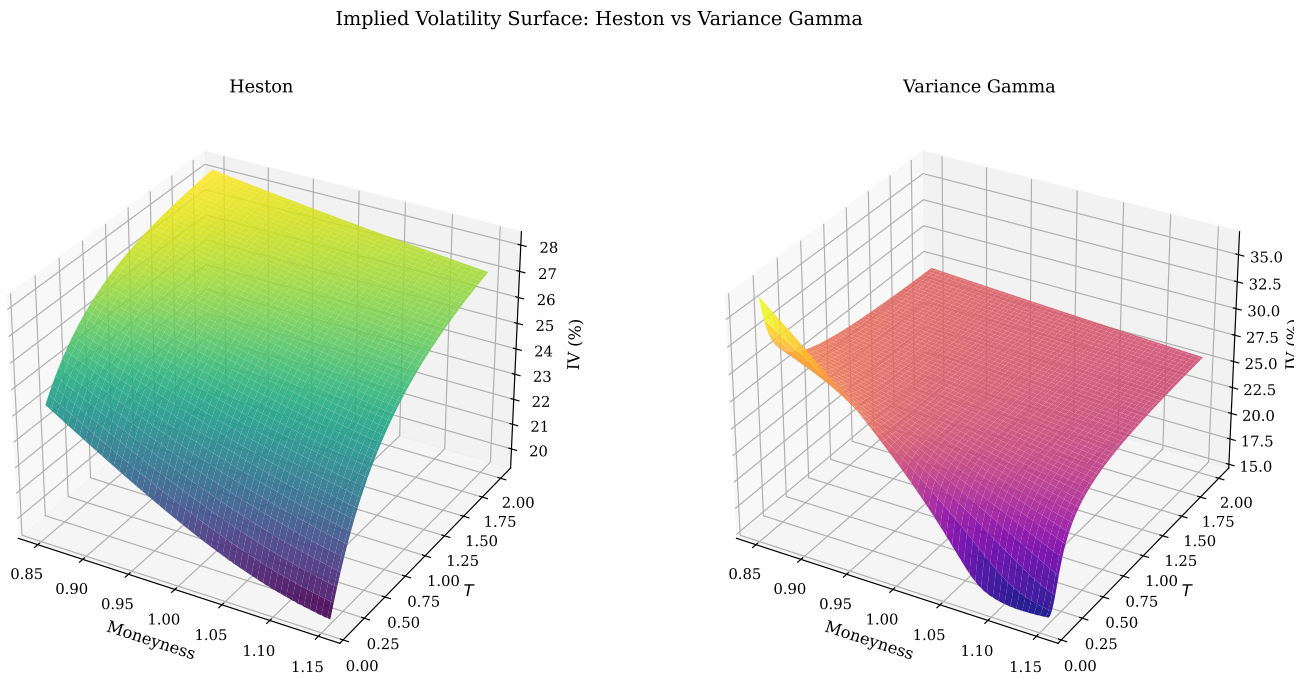

Figure 3. Implied Volatility Surfaces for the Heston (*left*, `viridis`) and Variance Gamma (*right*, `plasma`) models, calibrated on live SPY data and evaluated on a 100×100 grid spanning maturities $T \in [0.1, 2.0]$ and moneyness $K/S_0 \in [0.85, 1.15]$. The structural difference in surface shape highlights the distinct mechanisms: Heston generates a smooth leverage-driven skew via stochastic volatility, while VG produces a flatter, more symmetric surface driven by pure jump intensity and drift θ_{VG}.

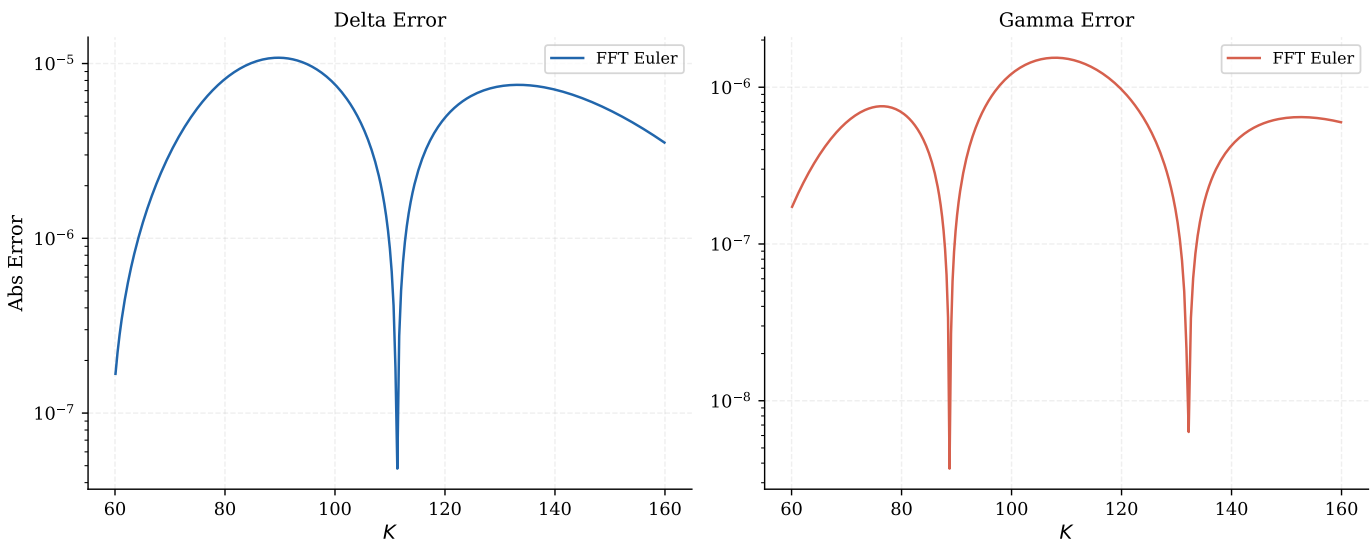

Figure 4. Validation of frequency-domain Greeks against the Black-Scholes closed-form solution. *Left:* absolute error in Delta; *Right:* absolute error in Gamma. Both are computed on a grid of $N = 8192$ points with $\alpha = 1.1$. The error remains near machine precision ($\sim 10^{-5}$–10^{-6}) across the entire ATM region, confirming the validity of the integral-derivative interchange.

$$\mathcal{E}_{disc} \approx \mathcal{O}\left(\eta^4 \max_{\upsilon \in [0,a]} |\psi_T^{(4)}(\upsilon)|\right) \tag{22}$$

Intuitively, to minimize $\mathcal{E}_{disc}$, the frequency step η must be extremely small. However, the FFT algorithm enforces the Nyquist relation $\lambda \cdot \eta = 2\pi/N$. A very small η artificially inflates the spatial log-strike spacing λ, leaving the region of interest (At-The-Money) severely under-sampled.

This introduces a fundamental trade-off: minimizing the discretization error in the frequency domain strictly compromises the spatial resolution in the strike domain. The standard mathematical workaround to tighten both bounds simultaneously without violating Nyquist is to exponentially increase the grid dimension (e.g., $N = 8192$). While the $\mathcal{O}(N \log_2 N)$ efficiency of the FFT makes this computationally feasible, expanding memory arrays exponentially is a sub-optimal compromise for ultra-low-latency pricing engines. The natural architectural evolution to decouple this restriction is the Fractional Fast Fourier Transform (FrFFT) [7], which introduces an independent fractional scalar β such that $\lambda \cdot \eta = 2\pi\beta/N$. While project time constraints precluded its implementation in the current build, integrating the FrFFT to fully neutralize the Nyquist trade-off remains the primary technical objective for the next iteration of the engine.

7. CONCLUSION

This technical project successfully architected, optimized, and validated a robust, low-latency option pricing engine using the Fast Fourier Transform. By decoupling the numerical integration framework from the stochastic dynamics, the engine seamlessly accommodates advanced characteristic kernels, successfully pricing complex phenomena via Heston, Merton Jump-Diffusion, and Variance Gamma models. Furthermore, the extension of the FFT methodology to analytically compute Greek surfaces fundamentally outpaces traditional numerical derivation. To further minimize latency and maximize performance, we adopted a hybrid architecture: the core numerical FFT loops and characteristic functions were natively executed in a compiled C++ extension via Pybind11, while the data and calibration components were retained in Python. This structural design bridges the gap between fast prototyping and high-frequency risk management pipelines. Future extensions of this architecture will explore the integration of the Fractional Fast Fourier Transform (FrFT) to decouple the spatial and frequency grids, further enhancing resolution control without exponential computational scaling.

■ ABOUT THE AUTHORS

This project was developed by **BlackSwan Quants** (BSQ), a quantitative finance student organization focused on advanced research and algorithmic trading engineering.
BSQ: https://www.linkedin.com/company/blackswan-quants/

A. REPOSITORY ARCHITECTURE AND CODE STRUCTURE

The project architecture is designed according to modern software engineering principles, maintaining a strict separation between high-level stochastic modeling, numerical integration, and low-latency compiled backends.

A.1. Core Python Interface: `fourier_options/`

This module serves as the installable Python library and the public API for the pricing framework.

- **`characteristic_functions.py`:** Contains the mathematical definitions of the market models (Black-Scholes, Merton Jump-Diffusion, Variance Gamma, and Heston). The functions strictly implement normalized underlying dynamics ($S_0 = 1$) to eliminate high-frequency aliasing during integration.
- **`fft_pricer.py`:** The algorithmic core implementing the Carr-Madan pricing framework. It evaluates the option chains simultaneously and leverages Euler's homogeneous function theorem to compute risk sensitivities (Delta, Gamma) analytically. The frequency-domain integrand modulation is performed in $\mathcal{O}(N)$, while the full sensitivity surface extraction operates in $\mathcal{O}(N \log N)$, systematically bypassing the numerical instability and heavy characteristic function re-evaluations inherent to finite-difference schemes.
- **`calibration.py`:** Dedicated to implied volatility surface extraction and market optimization. It features robust vector-mapped Newton-Raphson routines and minimization wrappers (e.g., Differential Evolution) for model calibration.

A.2. Low-Latency Engine: `engine/`

To resolve the computational bottlenecks of Python's internal iterations—which are critical when objective functions are evaluated thousands of times during non-convex optimization—the core numerical routines are pushed to a compiled layer.

- **`fft_pricer.cpp`:** A native C++ backend that houses the internal Radix-2 Cooley-Tukey algorithm, the robust root-finding solver (incorporating OTM/ITM switching to prevent Vega vanishing), and the core pricing loops.
- Utilizing `pybind11`, this module is compiled into the `cpp_pricer` extension. It directly exchanges numerical arrays with the Python interpreter, effectively bypassing the Global Interpreter Lock (GIL) and drastically accelerating execution times.

A.3. Research Pipeline and Reproducibility

The repository provides a streamlined environment for both exploratory quantitative research and strict academic reproducibility.

- **`generate_figs.py`:** The primary entry point of the project. It orchestrates the C++ engine to perform calibrations, compute models, and programmatically generate all the standardized PDFs (real market smiles, volatility surfaces, Greeks validation, and timing benchmarks) stored in the `paper/figs/` directory.
- **`notebooks/` and `tests/`:** A suite of Jupyter notebooks dedicated to exploratory sensitivity studies and algorithm visualization, supported by automated unit tests validating the implementation against known mathematical boundaries.

REFERENCES

[1] J. W. Cooley and J. W. Tukey, "An algorithm for the machine calculation of complex fourier series", *Mathematics of Computation*, vol. 19, no. 90, pp. 297–301, 1965.

[2] M. Broadie and P. Glasserman, "Estimating security price derivatives using simulation", *Management science*, vol. 42, no. 2, pp. 269–285, 1996.

[3] R. Storn and K. Price, "Differential evolution–a simple and efficient heuristic for global optimization over continuous spaces", *Journal of global optimization*, vol. 11, no. 4, pp. 341–359, 1997. DOI: 10.1023/A:1008202821328.

[4] P. Carr and D. B. Madan, "Option valuation using the fast fourier transform", *Journal of Computational Finance*, vol. 2, no. 4, pp. 61–73, 1999.

[5] D. Duffie, J. Pan, and K. Singleton, "Transform analysis and asset pricing for affine jump-diffusions", *Econometrica*, vol. 68, no. 6, pp. 1343–1376, 2000.

[6] P. Glasserman, *Monte Carlo Methods in Financial Engineering* (Applications of Mathematics). Springer, 2003, vol. 53.

[7] C. Chiarella, N. El-Hassan, and A. Kucera, "The evaluation of an american call option", *Quantitative Finance*, vol. 4, no. 1, pp. 41–58, 2004.

[8] R. Cont and P. Tankov, *Financial Modelling with Jump Processes*. Chapman and Hall/CRC, 2004.

[9] R. W. Lee, "Option pricing by transform methods: Extensions, unification and error control", *Journal of Computational Finance*, vol. 7, no. 3, pp. 51–86, 2004.

[10] L. B. Andersen and V. V. Piterbarg, "Moment explosions in stochastic volatility models", *Finance and stochastics*, vol. 11, no. 1, pp. 29–50, 2007.

[11] F. Fang and C. W. Oosterlee, "A novel pricing method for european options based on fourier-cosine series expansions", *SIAM Journal on Scientific Computing*, vol. 31, no. 2, pp. 826–848, 2008.

[12] R. Lord and C. Kahl, "Complex logarithms in heston-like models", *Mathematical Finance*, vol. 20, no. 4, pp. 671–694, 2010.

[13] A. Quarteroni, R. Sacco, and F. Saleri, *Numerical mathematics*. Springer Science & Business Media, 2014, vol. 37.